Line wrap

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Wisconsin Critical & Sensitive Area Assessment Report</title>

  <style>
    :root {
      --blue: #003f87; --blue2: #004d7a; --border: #e0e0e0;
      --text: #222; --muted: #666; --bg: #f5f5f5;
      --info-bg: #e8f0f7; --error-bg: #ffe8ec; --error: #b00020;
    }
    * { box-sizing: border-box; }

    body {
      margin: 0; background: var(--bg);
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
      color: var(--text); line-height: 1.45;
    }

    .header {
      background: linear-gradient(135deg, var(--blue) 0%, var(--blue2) 100%);
      color: #fff; padding: 1.5rem 2rem; box-shadow: 0 4px 12px rgba(0,0,0,.15);
    }
    .header h1 { margin: 0 0 .3rem; font-size: 1.8rem; }
    .header p { margin: 0; opacity: .95; }

    .container { max-width: 98vw; margin: 0 auto; padding: 1.5rem 2rem; }
    .section {
      background: #fff; border-radius: 8px; box-shadow: 0 2px 8px rgba(0,0,0,.1);
      padding: 1rem 1.25rem; margin-bottom: 1rem; border: 1px solid var(--border);
    }

    label { font-weight: 600; color: var(--blue); display: block; margin-bottom: .5rem; }
    select {
      width: 100%; max-width: 520px; padding: .6rem .7rem;
      border: 2px solid #d0d0d0; border-radius: 6px; font-size: 1rem; background: #fff;
    }
    .status-message {
      display: none; margin-top: .8rem;
      padding: .6rem .8rem; color: var(--blue); background: var(--info-bg);
      border-left: 5px solid var(--blue); border-radius: 4px;
    }

    .toolbar {
      display: flex; gap: .5rem; flex-wrap: wrap;
      align-items: center; margin-bottom: .6rem;
    }
    .btn {
      appearance: none; border: 1px solid #c9c9c9; background: #fff; color: var(--blue);
      padding: .45rem .7rem; border-radius: 6px; cursor: pointer; font-weight: 600;
    }
    .btn:hover { background: var(--blue); color: #fff; border-color: var(--blue); }
    .btn[disabled] { opacity: .5; cursor: not-allowed; }

    /* Scroll container (vertical + horizontal) — supports sticky header */
    .table-wrapper {
      max-height: 80vh;
      overflow-y: auto;      /* vertical scroll */
      overflow-x: auto;      /* horizontal scroll */
      position: relative;
      border: 1px solid #e0e0e0;
      border-radius: 6px;
      background: #fff;
    }

    table { width: 100%; border-collapse: collapse; font-size: .98rem; min-width: 980px; }
    thead { background: var(--blue); color: #fff; }
    th, td { padding: .6rem .7rem; border-bottom: 1px solid #e6e6e6; }
    th { position: relative; white-space: nowrap; cursor: pointer; }
    th .sort-ind { position: absolute; right: .6rem; opacity: .85; font-size: .85rem; }
    tbody tr:nth-child(odd) { background-color: #f8fbff; }
    tbody tr:nth-child(even) { background-color: #ffffff; }

    /* Sticky header cells */
    table thead th {
      position: sticky;
      top: 0;
      z-index: 10;
      background: #003f87;  /* match the thead color */
      color: #fff;
    }

    .error { color: var(--error); background: var(--error-bg); border-left: 5px solid var(--error);
      padding: .6rem .8rem; border-radius: 4px; margin-bottom: .6rem; }
    .info { color: var(--blue); background: var(--info-bg); border-left: 5px solid var(--blue);
      padding: .6rem .8rem; border-radius: 4px; margin-bottom: .6rem; }
    .muted { color: var(--muted); }
  </style>
</head>
<body>

  <div class="header">
    <h1>Wisconsin Critical &amp;amp; Sensitive Area Assessment Report</h1>
```

```
    <p>Select a Soil Survey Area (SSA). Sorted by Map Unit Name. Data from NRCS SDA Tabular API.</p>
  </div>

  <div class="container">
    <div class="section">
      <label for="ssa">Select a Soil Survey Area:</label>
      <select id="ssa">
        <option value="" style="display:none">(select a Soil Survey Area)</option>
        <!-- dynamic options will be loaded from SDA sacatalog -->
      </select>
      <div id="status" class="status-message"></div>
    </div>

    <div class="section">
      <div id="messages"></div>
      <div class="toolbar">
        <button class="btn" id="btnCsv" disabled>Export CSV</button>
        <button class="btn" id="btnXls" disabled>Export Excel (XLS)</button>
        <span id="rowCount" class="muted"></span>
      </div>
      <div id="tableContainer" class="table-wrapper"></div>
    </div>
  </div>

  <script>
    // ============ Utilities ============
    const SDA_URL = "https://sdmdataaccess.sc.egov.usda.gov/tabular/post.rest";

    const $ = sel => document.querySelector(sel);

    function showMessage(html, type = "info") {
      $("#messages").innerHTML = `<div class="${type}">${html}</div>`;
    }
    function clearMessage() { $("#messages").innerHTML = ""; }

    async function sdaPost(query, format) {
      const body = new URLSearchParams({ query, format }).toString();
      const res = await fetch(SDA_URL, {
        method: "POST",
        headers: { "Content-Type": "application/x-www-form-urlencoded" },
        body
      });
      if (!res.ok) {
        const text = await res.text();           // capture server text for diagnostics
        throw new Error(`SDA HTTP ${res.status} — ${text}`);
      }
      return await res.json();
    }

    // ============ Dynamic SSA dropdown ============
    async function loadSSAs() {
      const query = `
        SELECT areaname, areasymbol
        FROM sacatalog
        WHERE areasymbol LIKE 'WI%'
        ORDER BY areaname, areasymbol;
      `;
      showMessage("Loading SSA list…", "info");
      try {
        const data = await sdaPost(query, "json");           // pure data rows
        const table = Array.isArray(data.Table) ? data.Table : [];
        const sel = $("#ssa");
        sel.innerHTML = '<option value="" style="display:none">(select a Soil Survey Area)</option>';

        if (table.length === 0) {
          showMessage("No Wisconsin SSAs found.", "error");
          return;
        }
        for (const row of table) {
          const areaname   = row[0];
          const areasymbol = row[1];
          const opt = document.createElement("option");
          opt.value = areasymbol;
          opt.textContent = areaname;
          sel.appendChild(opt);
        }
        clearMessage();
      } catch (err) {
        console.error(err);
        showMessage(`Error loading SSAs from SDA: ${err.message}`, "error");
      }
    }

    // ============ YOUR QUERY (macro preserved; typos/entities fixed) ============
    function buildReportQuery(areasymbol) {
      return String.raw`
-- SDA macro:
~DeclareINT(@major)~
SELECT @major = 0; -- Enter 0 for major component, enter 1 for all components

DROP TABLE IF EXISTS #main_query;

SELECT
    l.areasymbol, l.areaname,
    m.muname, m.musym, m.mukey,
    x.slope_r, x.drainagecl,

    (SELECT TOP 1 comonth.flodfreqcl
```

```
     FROM comonth
     JOIN MetadataDomainDetail dd ON comonth.flodfreqcl = dd.ChoiceLabel
     JOIN MetadataDomainMaster dm ON dd.DomainID = dm.DomainID
     WHERE comonth.cokey = x.cokey AND dm.DomainName = 'flooding_frequency_class'
     ORDER BY dd.ChoiceSequence DESC) AS flodfreq,

    (SELECT TOP 1 comonth.pondfreqcl
     FROM comonth
     JOIN MetadataDomainDetail dd ON comonth.pondfreqcl = dd.ChoiceLabel
     JOIN MetadataDomainMaster dm ON dd.DomainID = dm.DomainID
     WHERE comonth.cokey = x.cokey AND dm.DomainName = 'ponding_frequency_class'
     ORDER BY dd.ChoiceSequence DESC) AS pondfreq,

    ISNULL((
     SELECT TOP 1 reskind
     FROM component AS c4
     INNER JOIN corestrictions ON corestrictions.cokey = c4.cokey
     WHERE c4.cokey = x.cokey AND reskind IN (
         'Densic bedrock','Lithic bedrock','Paralithic bedrock',
         'Cemented horizon','Duripan','Fragipan','Manufactured layer',
         'Petrocalcic','Petroferric','Petrogypsic'
     )
     GROUP BY c4.cokey, reskind, resdept_r, corestrictkey
     ORDER BY MIN(resdept_r) ASC, MIN(corestrictkey)
    ), 'No Data') AS first_restriction_kind,

    CAST (ROUND ((
     SELECT TOP 1 MIN (resdept_r) / 30.48
     FROM component AS c3
     INNER JOIN corestrictions ON corestrictions.cokey=c3.cokey
     WHERE c3.cokey=x.cokey AND reskind IN (
         'Densic bedrock','Lithic bedrock','Paralithic bedrock',
         'Cemented horizon','Duripan','Fragipan','Manufactured layer',
         'Petrocalcic','Petroferric','Petrogypsic'
     )
     GROUP BY c3.cokey, resdept_r
    ), 2) AS decimal (5,2)) AS rv_first_restriction_depth,

    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey) AS comp_count,
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.majcompflag = 'Yes') AS count_maj_
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.hydricrating = 'Yes') AS all_hydri
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.majcompflag = 'Yes' AND c.hydricra
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.majcompflag = 'Yes' AND c.hydricra
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.majcompflag <> 'Yes' AND c.hydricr
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.hydricrating <> 'Yes') AS all_not_
    (SELECT COUNT_BIG(*) FROM mapunit mu INNER JOIN component c ON c.mukey = mu.mukey WHERE mu.mukey = m.mukey AND c.hydricrating IS NULL) AS hydric_nu

    (SELECT TOP 1 coecoclass.ecoclassname
     FROM mapunit
     INNER JOIN component ON component.mukey=mapunit.mukey AND (CASE WHEN 1 = @major THEN 0 WHEN majcompflag = 'Yes' THEN 0 ELSE 1 END = 0)
     INNER JOIN coecoclass ON component.cokey = coecoclass.cokey
     WHERE mapunit.mukey = m.mukey AND ecoclasstypename LIKE 'Forage Suitability Groups'
     GROUP BY coecoclass.ecoclassname
     ORDER BY SUM(comppct_r) DESC) AS dom_cond_FSG_class,

    (SELECT TOP 1 coecoclass.ecoclassid
     FROM mapunit
     INNER JOIN component ON component.mukey=mapunit.mukey AND (CASE WHEN 1 = @major THEN 0 WHEN majcompflag = 'Yes' THEN 0 ELSE 1 END = 0)
     INNER JOIN coecoclass ON component.cokey = coecoclass.cokey
     WHERE mapunit.mukey = m.mukey AND ecoclasstypename LIKE 'Forage Suitability Groups'
     GROUP BY coecoclass.ecoclassid
     ORDER BY SUM(comppct_r) DESC) AS dom_cond_FSG_id,

    x.cokey
INTO #main_query
FROM dbo.legend AS l
INNER JOIN dbo.mapunit AS m ON m.lkey = l.lkey AND l.areasymbol = '${areasymbol}'
INNER JOIN (
    SELECT c1.cokey, c1.mukey, c1.compname, c1.comppct_r, c1.localphase, c1.drainagecl, c1.slope_r,
           ROW_NUMBER() OVER (PARTITION BY c1.mukey ORDER BY c1.comppct_r DESC, c1.cokey) AS rn
    FROM component AS c1
) AS x ON x.mukey = m.mukey AND x.rn = 1;

-- Final Select Statement
SELECT
    areasymbol AS [Area Symbol],
    areaname   AS [Area Name],
    muname     AS [Map Unit Name],
    musym      AS MUSYM,
    mukey      AS MUKEY,

    CASE
      WHEN slope_r >= 30               THEN 'Not Suitable'
      WHEN slope_r BETWEEN 12 AND 30   THEN 'Limited'
      WHEN slope_r < 12                THEN 'Suitable'
      ELSE 'Not Rated'
    END AS [Slope Class Rating],

    CASE
      WHEN drainagecl IN ('Poorly drained','Very poorly drained') THEN 'Not Suitable'
      WHEN drainagecl IN ('Somewhat poorly drained','Moderately well drained') THEN 'Limited'
      WHEN drainagecl IN ('Well drained','Somewhat excessively drained','Excessively drained') THEN 'Suitable'  -- comma fixed
      ELSE 'Not Rated'
    END AS [Drainage Class Rating],

    CASE
      WHEN flodfreq IN ('Very Frequent','Frequent') THEN 'Not Suitable'
      WHEN flodfreq IN ('Occasional','Rare')        THEN 'Limited'
```

```
      WHEN flodfreq = 'None'                       THEN 'Suitable'
      ELSE 'Not Rated'
    END AS [Flooding Class Rating],

    CASE
      WHEN pondfreq IN ('Very Frequent','Frequent') THEN 'Not Suitable'
      WHEN pondfreq IN ('Occasional','Rare')        THEN 'Limited'   -- IN fixed
      WHEN pondfreq = 'None'                        THEN 'Suitable'
      ELSE 'Not Rated'
    END AS [Ponding Class Rating],

    first_restriction_kind AS [Restriction Kind],

    CASE
      WHEN rv_first_restriction_depth < 50              THEN 'Not Suitable'
      WHEN rv_first_restriction_depth BETWEEN 50 AND 150 THEN 'Limited'
      WHEN rv_first_restriction_depth >= 150           THEN 'Suitable'
      ELSE 'Not Rated'
    END AS [Restriction Depth Class Rating],

    CASE
        WHEN comp_count = all_not_hydric + hydric_null THEN 'Nonhydric'
        WHEN comp_count = all_hydric                   THEN 'Hydric'
        WHEN comp_count <> all_hydric AND count_maj_comp = maj_hydric THEN 'Predominantly Hydric'
        WHEN hydric_inclusions >= 0.5 AND maj_hydric < 0.5             THEN 'Predominantly Nonhydric'
        WHEN maj_not_hydric >= 0.5 AND maj_hydric >= 0.5               THEN 'Partially Hydric'
        ELSE 'Error'
    END AS [Hydric Class Rating],

    dom_cond_FSG_id AS [WI FSG ID]
FROM #main_query
ORDER BY muname ASC, musym, mukey;
    `;
  }

  // ============ Table rendering & sorting ============
  function renderTable(headers, rows) {
    const table = document.createElement("table");
    table.id = "dataTable";

    const thead = document.createElement("thead");
    const trh = document.createElement("tr");
    headers.forEach((h, i) => {
      const th = document.createElement("th");
      th.textContent = h;                  // visible label (SQL alias)
      th.setAttribute("data-colname", h);  // raw label for export
      const ind = document.createElement("span");
      ind.className = "sort-ind";
      th.appendChild(ind);
      th.addEventListener("click", () => sortTable(table, i));
      trh.appendChild(th);
    });
    thead.appendChild(trh);

    const tbody = document.createElement("tbody");
    rows.forEach(r => {
      const tr = document.createElement("tr");
      r.forEach(c => {
        const td = document.createElement("td");
        td.textContent = c == null ? "" : c;
        tr.appendChild(td);
      });
      tbody.appendChild(tr);
    });

    table.appendChild(thead);
    table.appendChild(tbody);
    return table;
  }

  let sortState = { col: -1, dir: 1 }; // 1 asc, -1 desc
  function sortTable(table, colIdx) {
    const tbody = table.tBodies[0];
    const rows = Array.from(tbody.rows);
    const ths = table.tHead.rows[0].cells;

    if (sortState.col === colIdx) {
      sortState.dir = -sortState.dir;
    } else {
      sortState.col = colIdx;
      sortState.dir = 1;
    }

    Array.from(ths).forEach(th => th.querySelector(".sort-ind").textContent = "");
    ths[colIdx].querySelector(".sort-ind").textContent = sortState.dir === 1 ? "▲" : "▼";

    rows.sort((a, b) => {
      const av = a.cells[colIdx].textContent;
      const bv = b.cells[colIdx].textContent;

      const aNum = parseFloat(av);
      const bNum = parseFloat(bv);
      let cmp;
      if (!Number.isNaN(aNum) && !Number.isNaN(bNum)) {
        cmp = aNum - bNum;
      } else {
        cmp = av.localeCompare(bv);
      }
```

```
      return sortState.dir * cmp;
    });

    rows.forEach(r => tbody.appendChild(r));
  }

  function toCSV(tableEl) {
    const rows = [];
    const trsHead = tableEl.tHead ? tableEl.tHead.rows : [];
    const trsBody = tableEl.tBodies[0] ? tableEl.tBodies[0].rows : [];

    function esc(val) {
      const s = (val ?? "").toString();
      const needsQuotes = /[",\n]/.test(s);
      const escaped = s.replace(/"/g, '""');
      return needsQuotes ? `"${escaped}"` : escaped;
    }

    if (trsHead.length > 0) {
      const hdr = Array.from(trsHead[0].cells).map(th =>
        esc(th.getAttribute("data-colname") || th.textContent.replace(/[▲▼]/g, ""))
      );
      rows.push(hdr.join(","));
    }
    for (const tr of trsBody) {
      const cols = Array.from(tr.cells).map(td => esc(td.textContent));
      rows.push(cols.join(","));
    }
    return rows.join("\n");
  }

  function downloadBlob(content, filename, mime) {
    const blob = new Blob([content], { type: mime });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url; a.download = filename; document.body.appendChild(a); a.click(); a.remove();
    URL.revokeObjectURL(url);
  }

  function enableExports(tableEl, filenameBase) {
    const btnCsv = $("#btnCsv");
    const btnXls = $("#btnXls");
    btnCsv.disabled = false;
    btnXls.disabled = false;

    btnCsv.onclick = () => {
      const csv = toCSV(tableEl);
      downloadBlob(csv, `${filenameBase}.csv`, "text/csv;charset=utf-8");
    };

    btnXls.onclick = () => {
      const clone = tableEl.cloneNode(true);
      if (clone.tHead && clone.tHead.rows.length > 0) {
        Array.from(clone.tHead.rows[0].cells).forEach(th => {
          const ind = th.querySelector(".sort-ind");
          if (ind) ind.textContent = "";
        });
      }
      const html = `<html><head><meta charset="UTF-8"></head><body>${clone.outerHTML}</body></html>`;
      downloadBlob(html, `${filenameBase}.xls`, "application/vnd.ms-excel");
    };
  }

  // ============ Report runner ============
  async function runReport(areasymbol) {
    const container = $("#tableContainer");
    container.innerHTML = "";
    $("#rowCount").textContent = "";
    showMessage("Loading report…", "info");

    try {
      const sql = buildReportQuery(areasymbol);
      const data = await sdaPost(sql, "json+columnname");
      const table = Array.isArray(data.Table) ? data.Table : [];
      if (table.length === 0) {
        showMessage("No data available for this survey area.", "info");
        $("#btnCsv").disabled = true;
        $("#btnXls").disabled = true;
        return;
      }

      const headers = table[0];
      const rows = table.slice(1);
      const tbl = renderTable(headers, rows);
      container.appendChild(tbl);

      $("#rowCount").textContent = `${rows.length.toLocaleString()} rows`;
      enableExports(tbl, `WI_Critical_Sensitive_Assessment_${areasymbol}`);
      clearMessage();
    } catch (err) {
      console.error(err);
      showMessage(`Error loading SDA report for ${areasymbol}: ${err.message}`, "error");
      $("#btnCsv").disabled = true;
      $("#btnXls").disabled = true;
    }
  }

  // ============ Init ============
```

```
    document.addEventListener("DOMContentLoaded", () => {
      loadSSAs(); // dynamic list from SDA

      $("#ssa").addEventListener("change", (e) => {
        const areasymbol = e.target.value;
        if (!areasymbol) {
          $("#status").style.display = "none";
          $("#tableContainer").innerHTML = "";
          $("#rowCount").textContent = "";
          $("#btnCsv").disabled = true;
          $("#btnXls").disabled = true;
          return;
        }
        $("#status").textContent = `Selected: ${e.target.options[e.target.selectedIndex].text}`;
        $("#status").style.display = "block";
        runReport(areasymbol);
      });
    });
  </script>

</body>
</html>
```